Exact Top-K Selection on a Dataflow Many-Core: A Measured Design-Space Study

Anonymous Authors
Paper #XXX — IPDPS 2027 submission

***Abstract*—Exact top-k selection sits on the decode-time critical path of large language model serving: sampling selects k=32–64 winners from a padded vocabulary shard every token, sparse-attention indexers select k=512–2048 keys per layer, and mixture-of-experts gates fire every forward pass. GPU practice treats the problem as solved—count, partition, scatter, repeat—while accelerator studies either approximate the selection or never measure it. We present a measured design-space study of exact top-k on a Tenstorrent Blackhole p150a, a dataflow many-core with 130 Tensix workers on a 13×10 mesh NoC, software-managed SRAM, and neither atomics nor scatter in the compute path. Single-ingredient silicon microbenchmarks show why GPU radix-select economics do not transfer: exact SFPU counting yields 1 bit per 2.0 cycles/vector, a count-to-decision rendezvous costs 81 cycles, and materialization—not counting—is the load-bearing gap (dense emission 13.0 cycles/element against a 0.5 bar). The design space instead rewards parallelizing the data-oblivious comparison network: a column-parallel log-tree operator governed by a validated cost model $2\lceil C/P\rceil + \lceil \log_2 P\rceil$. The operator measures 34.3 µs at k=2048, N=65,536 (bf16) on 32 cores—18,401× over the pre-study stock path—and scales monotonically to 23.3 µs at k=512, N=262,144 on 104 cores. A distribution-free chunk-skip law $P(\text{skip}) \approx e^{-K/(c+1)}$ with a compile-time $K/4$ gate adds 1.82× (−45.1%) at k=32, N=65,536. End to end, the user-facing `ttnn.topk` call at k=2048, N=65,536 improves from 631.5 ms to 51.5 µs—12,265×—and the surviving gap to the vendor roofline spans 9.7–29.9× across all 24 measured cells. The study is honest about its own dynamics: the incumbent bitonic engine absorbed the study's routing and placement fixes and outran the radix-select model it was measured against. We also report the first public characterization of the Tensix datapath's numeric semantics: bf16 canonicalization, a native sign-magnitude total order including NaN, and a sampled, aliased packer exponent histogram.**

***Index Terms*—top-k selection, dataflow architectures, network-on-chip, many-core, Tenstorrent Blackhole, performance characterization**

I. Introduction

On a Tenstorrent Blackhole p150a, Qwen3 decode sampling under tensor parallelism 4 shards a 151,936-entry vocabulary into 37,984 logits per device and pads the shard to 65,536—the one power-of-two width that fails the shipping bitonic engine's W<65,535 routing gate. The call lands on a single-core linear factory that walks logits at ≈137 ns/element: 9,587 µs per sampled token for a k=32 selection, while the other 129 cores idle. The cliff is not a corner case. At the sparse-attention indexer shape k=2048, N=65,536 (bf16), the stock `ttnn.topk` path measured 631.5 ms per call at the start of this study; the same call on the same silicon now measures 51.5 µs—12,265×—with no new hardware. This paper argues one sentence: exact top-k on a commercial NoC-mesh dataflow chip sat four orders of magnitude off its achievable floor, and a measured design-space study—not a clever new algorithm—closed the gap.

Blackhole is a dataflow many-core: 130 Tensix workers on a 13×10 grid, each with five RISC-V processors, a 32-lane SIMD unit (SFPU), and software-managed SRAM, joined by a 2D NoC [1]. Every fast GPU top-k mechanism leans on primitives this machine lacks — atomic histograms, scatter compaction, grid-wide synchronization [2], [3], [4] (§II-A).

The missing primitives are not one vendor's oversight. AI accelerators across the industry traded coherent shared memory and atomics for mesh bandwidth and software-managed SRAM, because matrix multiplication rewards the trade. Selection is the canonical data-dependent workload the trade punishes, and LLM decoding keeps placing it on the critical path (§II-A fixes the data contract) [5], [6], [7], [8]. Nobody has measured what the trade costs here: no sorting or selection publication exists for the dataflow-accelerator class (§VII), and TPU studies escape into approximation [9], [10]. Mesh-selection theory proved step bounds on abstract arrays [11], [12], but returned neither values-with-indices nor silicon measurements.

We measured the design space ingredient by ingredient instead of betting on one kernel. The study decomposes every exact top-k engine along three questions—winner identification, data touched per pass, decision cost between passes—and holds Blackhole against each with single-ingredient microbenchmarks (§III). The answers overturn the GPU playbook: counting is narrow, decisions are affordable, and materialization is the wall, so radix select degenerates to threshold bisection and loses. The space instead rewards parallelizing the data-oblivious comparison network as a column-parallel log-tree operator (§IV); the one data-dependent mechanism that survives without scatter is a chunk-skip rule, sound by construction and gated at compile time by a distribution-free rank-statistic law.

The study is candid about its own dynamics. The incumbent bitonic engine absorbed the study's routing and placement fixes and finished at 34.4 µs on the anchor cell (k=2048, N=65,536)—inside the uncertainty band of the radix-select model priced from our own microbenchmarks; a pre-registered stop rule closed that track (§III). The same candor governs the headline: §VI-F attributes the orders of magnitude mostly to routing around the single-core fallback, and the algorithm to a measured 1.4× over the strongest eligible stock configura-

tion plus its measured tree, placement, and skip terms. The surviving gap to the vendor roofline is 9.7–29.9× across all 24 measured cells (§VI).

This paper contributes:

- **Quantification**, via single-ingredient silicon microbenchmarks, of why GPU radix-select economics do not transfer to a NoC-mesh dataflow core: counting is narrow, decisions are affordable, and materialization—not counting—is the load-bearing gap (§III).
- **Design and measurement** of a column-parallel log-tree top-k operator—semaphore-tree rendezvous without atomics or global synchronization—with the validated cost model $2\lceil C/P \rceil + \lceil \log_2 P \rceil$ and a cost-optimal rectangle embedding: 34.3 µs at k=2048, N=65,536 on 32 cores (18,401× over the pre-study stock `ttnn.topk`) and monotone scaling to 23.3 µs at k=512, N=262,144 on 104 cores (§IV, §VI). To our knowledge, the first *measured* full top-k (values and indices) on commercial NoC-mesh silicon (§VII).
- **Derivation and silicon validation** of a distribution-free chunk-skip law $P(\text{skip}) \approx e^{-K/(c+1)}$ for streaming bitonic cascades, with a soundness proof, a compile-time $K/4$ gate, and a calibrated pre-build forecast that eliminated the losing variant; the surviving variant measures 1.82× at rows=2, k=32, N=65,536 (§IV).
- **First public characterization** of the Tensix compute datapath's numeric semantics and sorting-relevant primitive costs: bf16 canonicalization (NaN→Inf, $-0 \to +0$, subnormal→ $+0$), a native sign-magnitude total order including NaN, a 1-in-8-sampled and 32-bin-aliased packer exponent histogram, and measured synchronization floors (§V).

Every performance cell behind these claims is gated bit-exact against a host golden and every headline number traces to a committed measurement artifact; the ledger and canonical harness will be released, and §VIII discloses the agentic campaign methodology.

II. Background

A. Problem Statement and Data Contract

The object of study is exact top-k selection: given one or more rows $x \in \text{bf16}^N$ resident in device DRAM, return the k largest elements of each row — values *and original indices*, exact rather than approximate, sorted descending. Under `stable=false`, ties may resolve to any valid winner set but must be deterministic for a fixed input. A 62-cell differential contract suite pins what the shipping engines actually promise: the returned values are the exact top-k multiset of the input *as canonicalized by the datapath* (bf16 specials are rewritten on entry; Section V pins the rules) under the hardware's native sign-magnitude total order; every index is unique and satisfies $x[\text{index}] = \text{value}$ bit-exactly (short rows emit a documented sentinel index); repeated launches return bit-identical outputs. bf16 is the datatype at every surveyed production call site; fp32 traverses bit-exactly. Production adds three contract variants: indices-only versus (values, indices) outputs, multi-row batches, and a valid-length prefix that masks a stale tail.

Where does x come from, and where do the results go? The row is always the upstream operator's output — an lm-head matmul, an attention-indexer score, or an expert-gate projection — DRAM-interleaved in the producer's layout; results return to DRAM. Three consumer classes cover every surveyed call site (Table III): token sampling needs values and indices at k=32 over vocabulary shards of N=16–65K every decoded token; sparse-attention KV gather needs indices only at k=512–2048 over contexts up to 1M, every layer; mixture-of-experts dispatch needs k=4–16 winners over 128–512 experts every layer. A selection engine's real cost therefore includes the layout conversions between producer, engine, and consumer formats — why Section VI prices end-to-end composites, never bare kernels.

The machine class sets the difficulty. CPU selection — vectorized quickselect of the `x86-simd-sort`/vqsort family [13], [14] — rides coherent caches and branch predictors: branchy, data-dependent control flow is cheap and the scalar pivot logic is free. GPU selection rides shared-memory atomic histograms, scatter compaction, and grid-wide synchronization — Section III prices each of these affordances on this silicon. Blackhole has none of them: no cross-core atomics, no scatter or compaction in the compute path, no global barrier — synchronization is NoC semaphores over software-managed SRAM, the 32-lane SFPU counts 1–3 bits per pass, and the fast path is a fixed, data-oblivious instruction stream, so every data-dependent decision costs a measured 81-cycle rendezvous.

B. Blackhole and the Tensix Core

The Tenstorrent Blackhole p150a exposes 130 programmable Tensix workers on a 13×10 grid, joined by a 2D network-on-chip (NoC) [1]; the count is a property of this unit (harvesting varies; no result hardcodes it). Each core owns ≈1.5 MB of software-managed L1 SRAM [1]; there is no cache hierarchy and no coherence protocol.

Five RISC-V processors share that L1: two move data over the NoC, and three compute as an *unpack→math→pack* pipeline. The math stage owns the SFPU, a 32-lane SIMD engine, and stages its output in the Dst register file that pack consumes. An operator is therefore three-to-five cooperating programs per core, synchronized through circular buffers in L1. Sections III and V price these units one at a time; Section IV spends them.

C. The Incumbent Bitonic Engines

The vendor stack routes `ttnn.topk` through a bitonic local sort in the low-level kernel library (`topk_local_sort`) [15], [16]. The bitonic leaf sorts a window of M elements resident in Dst, where M snaps upward from the requested k: $k \le 512$ gives M=512, $k \le 1024$ gives 1024, and larger k gives 2048. We fix the paper's notation here: a row holds N elements, split into

$C = N/M$ chunks; P cores cooperate on one row; K is the user's k. All later sections inherit these symbols.

Before this study, two engines shipped. A multi-core bitonic path accepts only rows with power-of-two width, $W < 65{,}535$, $k \leq 64$, and $N \geq 4096$. A single-core linear factory catches everything else at ≈137 ns/element: 695 µs at W=5,000 (k=32), growing to 17.95 ms at W=65,536 (k=64). Between the two engines sits a measured cliff. At the $W < 65{,}535$ gate, a $k \leq 32$ row jumps from 106.7 µs (multi-core, 65 cores, N=8,192) to 9.49 ms (one core, N=65,534) — ≈89× more time for 8× more elements. The cliff is architecture context here; Section VI owns the fix. The bitonic engine itself is data-oblivious — one corner of the design space Section III maps and prices.

Exact GPU top-k, by contrast, is a mature family, each branch leaning on an affordance Blackhole must reprice: radix/bucket select on atomic histograms plus scatter compaction [2], [17], [4]; WarpSelect's per-lane threshold queues [18]; threshold-guess counting [7]; TPU-KNN's matmul-shaped approximation [9], [10]. Section III prices each on silicon.

III. Why GPU Selection Economics Do Not Transfer

GPU radix select wins because its hardware sells wide exact counting and cheap materialization [2], [3], [17], [4]; Blackhole quotes different prices. This section decomposes exact selection into three orthogonal questions, prices each ingredient on silicon, and locates the one that kills the family: materialization, not counting or synchronization. The (A)–(F) vocabulary defined here carries through Sections IV–IX.

A. Three Questions, Six Answers

Every exact top-k engine answers three independent questions. **Q1 — winner identification:** either **(A)** a comparison network — a fixed, data-oblivious schedule of compare–exchanges (bitonic sort/merge, WarpSelect [18]) — or **(B)** counting/partitioning, which counts elements beyond a boundary and *decides* where to look next. **Q2 — data touched per pass:** either **(C)** full-data passes every time, or **(D)** shrinking candidate sets, by scatter compaction or count-guided skipping. **Q3 — decision cost between passes:** **(E)** nothing, for oblivious schedules; or **(F)** a synchronization rendezvous that carries the count to a scalar decision-maker and redistributes the verdict.

GPU radix select answers (B)+(D)+cheap-(F): 8–12-bit atomic histograms, scatter compaction, grid-sync decisions (Section II) [2], [4]; the shipping Blackhole engine answers (A)+(C)+(E) — ≈2 cycles/element on every pass, zero decisions. Can Blackhole move to (B)?

B. The Ingredients, Measured

We price the ingredients one at a time; all costs are slope-measured cycles, gated bit-exactly against host goldens (§VI-A).

Wide exact counting (Q1-B). The 32-lane SFPU counts exactly but narrowly: one predicate bit per pass at 2.0000 cycles/vector, three bits at 3.0, riding additively on the 3.94 cyc/vec unpack floor (+2.42/+4.52 in-stream; 2.44/4.53 isolated). A GPU pass resolves 8–12 bits [2], [4]; an SFPU pass resolves 1–3, so a radix engine on this datapath collapses into a bisection. The scalar cores are no rescue: a 256-bin RISC histogram runs 7.56–7.90 cycles/element against a 3.02 pure-load floor (8.23 with 4-way sub-histograms; 15.5–19.9 L1-resident).

Cheap decisions (Q3-F). Decisions measure 81 cycles each at best: a `tensix_sync`, a cross-lane SFPU fold, and one memory-mapped read of the DST register file (folding is mandatory — raw-lane reads cost 756–773 cycles, ≈9× worse). At 81 cycles ≈ 60 ns[1] and ≈14 decisions per row (Section III-C), decisions are affordable — not the blocker.

Materialization (Q2-D). Dense survivor emission on a RISC core measures 12.97–17.81 cycles/element against a ≈0.5 cycles/element bar for compaction to pay — dead. The SFPU has no scatter path at all. The sole survivor, the packer's zero-compressed output stream with a skip-zero RISC consumer, costs 0.63 cycles per original element: ≈3× under one bitonic leaf pass but 2–5× above GPU-paper compaction — and it is consumer-only, measured on uniform input: no device-side decoder exists, and the composed producer+consumer cost was never measured — reopening condition #1 in Section IX.

C. Degeneration to Threshold Bisection

Without (D), every counting pass is strictly additive to the bitonic finish that must still run; what survives of (B) is threshold bisection on the key space, monotone because sign-magnitude keys order the probes (Section V). Over 57 input cells — random, clustered, and all-equal distributions across seeds, a $K \in \{31, 32, 33\}$ straddle, ties, and ±0/Inf/NaN/denormal specials — bisection to the exact K-th threshold costs a median of 14 decisions and 2,313 cycles per 1,024-element row, worst case 17 on the clustered and tied inputs key-space bisection predicts; the certification invariant $C_{\mathrm{gt}} < K \leq C_{\mathrm{gt}}+C_{\mathrm{eq}}$ held field-exactly on every row. Per decision, ≈165 cycles — one count pass (≈64–70), one rendezvous (81), loop overhead: the (F) matrix composes as priced. At ≈1.7 µs per row the exact arm is honest — Floyd–Rivest economics [19], not radix economics. The family fits Blackhole only in its degenerate corner, (B)+(C)+(F).

D. Why the Track Closed

A selector composed from the measured components — packer pre-filter, count passes, rendezvous, materialization, final K-sort — prices at ≈21–30 µs per large cell. The pre-registered stop rule, written before any measurement, demanded an incumbent ≥2× the model to justify building. The incumbent finished the study at 34.4 µs (k=2048, N=65,536)

[1]Cycle→time conversions assume a 1.35 GHz busy clock; the clock under load has not been captured (idle 800 MHz measured). Section VI states the caveat once for the whole paper.

Algorithm 1 Log-tree top-K with dominance chunk-skip

Require: row $= C$ chunks of M; P slices (Eq. 1); row-parallel mode: all C chunks on one core, no tree
Ensure: exact top-K values and original indices, sorted descending
for all slices p **in parallel do**
  $W_p \leftarrow \text{bitonic\_sort}(\text{chunk}_0)$ ▷ M-wide window resident in Dst
  for $c = 1, \ldots$ (slice's remaining chunks) **do**
    if row-par. $\wedge\ c \geq \max(2, K/4) \wedge \max(\text{chunk}_c) < W_p[K]$ **then**
      skip ▷ sound: $W_p[K] \leq v_K$ (§IV-C); gate: Eq. 2
    else
      $W_p \leftarrow$ top-M of $\text{merge}(W_p, \text{bitonic\_sort}(\text{chunk}_c))$
    end if
  end for
end for
for level $\ell = 1, \ldots, \lceil \log_2 P \rceil$ **do** ▷ NoC semaphore tree; no atomics, no global barrier
  for all surviving pairs (p, q) **in parallel do**
    q sends W_q to p ▷ one M-wide window; $P-1$ total
    $W_p \leftarrow$ top-M of $\text{merge}(W_p, W_q)$ ▷ losers discarded in place
  end for
end for
root emits $W[1{:}K]$ values and indices

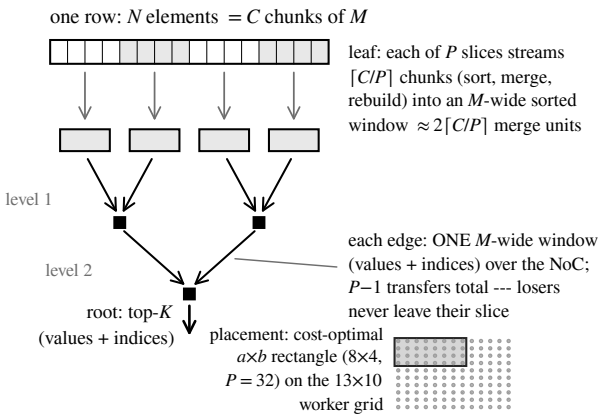

Fig. 1. Column-parallel log-tree top-k operator.

— inside the model's uncertainty band — because the study's own first gate produced two structural fixes (a small-k routing repair and a core-count cap raise, Sections IV and VI) that made the incumbent 15–27% faster while the selector was being priced. A selector that must outrun its own evaluation's side effects is not a production bet; the track closed by its own rule. The machinery was banked: the rendezvous and fold primitives price Section IV's chunk-skip decision; the floors join Section V.

IV. A Log-Tree Top-K Engine for a NoC Mesh

The map of Section III leaves two live moves: parallelize the data-oblivious corner (A)+(C)+(E) — split the row across cores, pay a log-depth rendezvous instead of a shared-memory partition — and import the shrinking axis (D) in the only form that survives without scatter: skip work, never move data. Sections IV-A–IV-B build the operator (`topk_large_indices`); IV-C–IV-E build chunk-skip and the forecast that eliminated its losing variant pre-build.

A. Operator Structure

Figure 1 shows the operator on one row and Algorithm 1 its control flow, in §II-C's notation: the leaf window M snaps to the LLK sort width and the row splits into $C = N/M$ chunks. The program places P slices on a rectangle of cores; each slice streams $\lceil C/P \rceil$ chunks through a bitonic leaf that sorts the arriving chunk, merges it into a resident M-wide descending window, and rebuilds the window order. A $\lceil \log_2 P \rceil$-level in-place merge tree — the mesh-shaped sibling of FPGA merge-tree sorters [20], applied to selection — then folds the P windows to a root core, which emits the top-K values and indices; every level halves the live cores and merges two M-wide sequences; Fig. 1 annotates the consequence — $P-1$ window transfers total, losers never leave their slice.

The synchronization fabric is deliberately austere. Tree levels map one-to-one onto NoC semaphore levels; each core owns a single receive circular buffer sized for two M-wide sequences; the writer kernels extend to seven partner slots (P=128). The operator uses no atomics, no global barrier, and no dedicated DRAM scratch (§II-A) [2], [4]. The tree replaced a serial merge-chain ancestor that folded slices one at a time: at fixed core counts the swap cut k=2048, N=65,536 from 69.6 µs to 41.9 µs (1.66×) and k=512, N=262,144 from 64.2 µs to 32.0 µs (2.01×), and it made runtime monotone in P — the property the cost model exploits.

B. Cost Model and Rectangle Embedding

The factory chooses P by minimizing a two-term model in leaf merge units:

$$\text{cost}(P) = 2\lceil C/P \rceil + \lceil \log_2 P \rceil,\ P^\star = \underset{\substack{P \in [2,\, \min(C,128)] \\ P = a \times b \text{ embeds}}}{\arg\min} \text{cost}(P), \tag{1}$$

with ties resolved toward fewer cores. The factor of two prices the leaf honestly: every chunk after the first costs ≈2 merge units (sort, merge, rebuild) while chunk 0 costs one. The unit constant is measured, not assumed: 1.46 µs per merge unit at M=512 and 5.51 µs at M=2048 (≈1,966 and ≈7,440 cycles[2]). The leaf term is mesh-selection theory's multi-packet regime [11], [12], [21] with measured constants; the log term is the mesh's rendezvous price.

Eq. 1's embedding constraint has teeth on a harvested mesh. An earlier full-rows-only fit silently clamped P: at k=2048, N=65,536 it settled for 13×2 = 26 cores while an 8×4 = 32-core rectangle sat free. Searching all cost-optimal rectangles instead wins 8–27% at the op level across the re-pinned cells: 41.9→34.4 µs (26→32 cores), 58.6→49.6 µs (52→64), 32.0→23.3 µs (52→104), and 15.0→13.1 µs (52→64) — the embedding search is part of the cost model, not a detail below it. Section VI-C defends the model against the 15-point P-sweep (Fig. 3), including the bump it predicts.

C. Chunk-Skip: Mechanism and Soundness

Ingredient (D) — shrinking the touched data — survives on this machine only if nothing has to move. Chunk-skip is that import: a per-chunk predicate that decides whether the

[2]Cycle forms of Tracy-measured times assume a 1.35 GHz busy clock; the clock under load was not captured (idle 800 MHz measured). Section VI states the policy; all cycle→time conversions in this section carry it.

leaf pipeline needs to run at all. Let T be the running K-th largest survivor, read from the resident sorted window. The rule is strict — skip chunk c iff $\max(\text{chunk}_c) < T$, the guard of Algorithm 1; chunk 0 always seeds the window. T is monotone nondecreasing because merges only improve the window; boundary ties ($\max = T$) are never skipped; and the decision is a pure function of the input, so the output stays deterministic.

Soundness admits a short argument: at most $K-1$ elements ever exceed the row's final K-th largest value v_K, so $T \le v_K$ at all times; any member x of an exact top-K set satisfies $x \ge v_K \ge T$, so the strict test never fires on x's chunk, and x, sitting within the top-$K \le$ top-M, survives every M-wide merge to the root. The released kernel source carries the full 288-line proof as a header comment. The GPU running-threshold family contrasts on all three counts (§VII): WarpSelect is element-granular with no analytic law [18]; GVR must count, verify, and refine after a misprediction [7]; chunk-skip prunes at M-element granularity, sound by construction — no verification pass exists to pay for — with a distribution-free law, compile-time gate (§IV-D), and pre-build forecast (§IV-E). Adversarial validation backs the proof: a 36-check battery (all-equal, ascending, last-chunk winners, replicated boundary ties) is bit-exact gated and ungated, and a 20-launch hang battery passed twice.

D. The Skip Law and Its Compile-Time Gate

How often the rule fires is a rank statistic, not a distribution property. For i.i.d. inputs, chunk $c{+}1$ is skippable iff the top-K of the first $(c{+}1)M$ elements all lie in the first cM:

$$\Pr[\text{skip at } c] \;=\; \binom{cM}{K} \Big/ \binom{(c+1)M}{K} \;\approx\; e^{-K/(c+1)}, \quad (2)$$

identical for normal, uniform, and softmax-shaped inputs because only ranks enter. Fig. 2 plots both regimes: the law is unforgiving at short streams — a conservative $K{=}512$ threshold skips with probability 2×10^{-307} at $c{=}1$ and reaches only 1.8% at $c{=}128$, while an aggressive user-$K{=}32$ threshold — sound by the argument of Section IV-C even though the resident window is M-wide — reaches 37% at $c{=}32$ and 78% at $c{=}128$.

The law hands us the gate. Below $c = K/4$ the skip probability is under $e^{-4} \approx 1.8\%$, so the compile-time gate sets the first tested chunk to $\max(2, K/4)$ — a derivation from Eq. 2, not a tuned constant (the floor of two: the threshold address needs §V's post-rebuild window layout, which exists only after the first merge). The gate zeroes the measured ungated overhead (+6.8% at $K{=}512$ over 128 chunks, +1.8% at $K{=}1536$ over 25 chunks) while forfeiting under one expected skip per row. The decision itself is cheap: ≈150 ns per tested chunk at $M{=}512$ and ≈350 ns at $M{=}2048$, derived from the ungated A/B (126 tested chunks cost +18.9 µs) — against ≈2.0 µs and ≈5.4 µs saved per skipped chunk.

E. Forecast Before Build: The Variant We Did Not Build

Before any kernel was written, an exact host-side simulation — bf16 sign-magnitude streaming, stimulus matched to the sweep harness, calibrated to reproduce all five pinned measured cells to ±0.0 µs by construction — priced chunk-skip inside the column-parallel tree. The verdict was a no-go with the weight of a result: 0.00% skip on every realistic distribution on every cell, a predicted +0.1–0.4 µs pure regression from the test overhead. The cause is structural: *the rectangle split and the skip law fight over the same resource, stream length c* — the cost-minimizing split caps each slice at $\lceil C/P \rceil = 1$–5 chunks while Eq. 2 needs $c \gtrsim K/4$. The split already won.

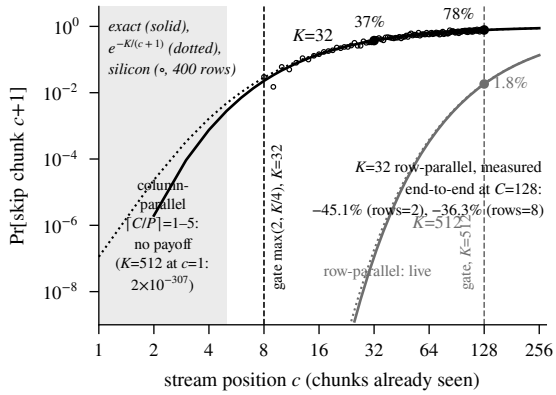

Fig. 2. Skip law (Eq. 2): exact binomial (solid), $e^{-K/(c+1)}$ (dotted), and measured on-device per-position skip rates (dots: 400 rows, $K{=}32$; §VI-D); end-to-end deltas annotated.

The build went where the law points: the row-parallel path, each core streaming all C chunks of its own row — long streams, small K, exactly the shapes the stock small-k routing ships here. The forecast's redirect predicted 54.3% skip at $K{=}32$ on a 128-chunk stream. At rows=2, $K{=}32$, $N{=}65{,}536$ the measured time falls 1.82× (−45.1%), and at rows=8 by 1.57× (−36.3%) — both on 130-core programs whose active cores number $\min(\text{rows}, 130)$, so the rows=2 cell exercises two cores, not the grid. Cells where the law forbids a win are gate-protected; Section VI-D quotes the A/B, the guard cell, and the on-device telemetry that confirms the law. The forecast said no; we did not build that variant.

V. Characterizing the Tensix Datapath for Selection

Selection exercises corners of a datapath that matmul studies never touch: NaN ordering, signed zeros, operand routing, and the cost of getting one count off the SIMD unit. This section banks those facts as an archival reference in the microbenchmark-dissection genre of Jia et al. [22]; no prior Tensix study reports datapath numerics or selection-primitive costs (§VII). Table I anchors the section. Cycle figures are slope-measured with a 2.000× control tripwire (§VI-A); the section stays in cycles, so no busy-clock conversion caveat applies.

A. Numeric semantics

An identity-op probe shows the bf16 compute datapath rewrites specials before any operation touches them — Table I, row 1, gives the full rules — while index positions survive and

TABLE I
MEASURED TENSIX DATAPATH LAWS FOR SELECTION.

Property	**Measured behavior** *(probe)*	**Consequence for selection**
bf16 canonicalization	NaN (any payload) $\rightarrow$ same-sign Inf; $-0 \rightarrow +0$; $\pm$subnormal $\rightarrow +0$ (sign dropped); index positions preserved; fp32 passes bit-exactly *(identity-op probe; 62-cell contract suite with JSONL divergence ledger)*	reference models must canonicalize first; enables exact (never-PCC) correctness gates
Hardware total order	$-\mathrm{NaN} < -\mathrm{Inf} < \cdots < -0 < +0 < \cdots < +\mathrm{Inf} < +\mathrm{NaN}$, native in `SFPGT`, `SFPSWAP`, and the packer threshold unit; $+$NaN survives the threshold, $-$NaN is zeroed *(failing $\pm$NaN differential test)*	the compare order *is* the radix-key order; no float$\rightarrow$uint bit-flip map needed
Exponent histogram: sampling	enable cost zero ($25.175 \rightarrow 25.104$ cyc/tile); samples 1-in-8 positionally ($p \bmod 64 < 8$: exactly 128 increments per 1024-datum tile, format-independent) *(38/38 functional + 6/6 perf; marker patterns + mod-8 phase sweep)*	exact-count designs dead on arrival; survives only as a sampled threshold seed
Exponent histogram: aliasing and traps	bins alias exponent $\wedge$ 31 (32 bins; exp $127 \equiv 159$); 8-bit counters saturate at 255; sign-blind; `WhichPackers` ignored in read modes 6/7 (silent $4\times$ overcount); `CLREXPHIST` (1 cyc) does not fence in-flight packs ($\approx$39-count leak) *(same bench)*	consumers must debias $4\times$, guard saturation, and handle sign upstream
Threshold filter	$T \geq 0$: compare-and-zero during pack at 4.034 cyc/vec vs. the bench's 3.855 local floor ($+4.6\%$); $T < 0$: documented UB, acts as $\lvert T \rvert$ rounded up to the next power of two (mantissa ignored) *(negative-filter bench)*	free pre-filter for non-negative data; signed data falls back to `SFPGT`(SET_VD)+`SFPAND`, 2 issues/vec — the ISA floor
Dst window layout	$\mathrm{word}(r) = (r \bmod (M/16)) \cdot 16 + \lfloor r/(M/16) \rfloor$: descending window column-major over $M/16$ physical rows; exhaustive at $M{=}512/1024/2048$ *(full-region Dst dumps vs. distinct-monotone input)*	the running-threshold read of §IV's chunk skip lands on the right word
`SFPSWAP` operand order	mod1$=$1 routes max$\rightarrow$VC, min$\rightarrow$VD; the in-tree comment reads backwards *(decision tracer, $-\infty$ accumulation)*	cross-lane max folds accumulate the correct operand
Count / rendezvous floors	count rate 2.0000 cyc/vec exactly; fp32 unpack-to-dest floor 3.938 cyc/vec; rendezvous 81/101/98 cyc (`tensix_sync` / semaphore / Dst sentinel); no-fold 756–773 cyc; MMIO $\approx$10 cyc/word; PassSync $\geq$25.1 cyc *(counting + rendezvous benches, MATH_ISOLATE two-point slope)*	sets §III's counting ceiling; fold-on-SFPU before any RISC read is mandatory
`SFPLOADMACRO` co-issue law	Load$+$Simple$+\{$MAD $\mid$ Store$\}$ maps at 1.000–1.003 cyc/vec (three sub-units co-issue free); reductions need $\geq$2 issues (`SFPIADD` not macro-schedulable) *(paired arms with mutation controls)*	counting cannot beat 2 cyc/vec; the law behind the shipped merge/step wins

fp32 passes bit-exactly. Vendor ISA documentation corroborates the Dst behavior [23]. This rule is the precondition for every correctness gate in the paper (§II-A).

A failing differential test on $\pm$NaN specials exposed the second law: the comparators implement the sign-magnitude total order of Table I natively in `SFPGT`, in `SFPSWAP`, and — measured here, absent from vendor documentation — in the packer's threshold unit, which passes $+$NaN and zeroes $-$NaN. GPU radix sorts build this order in software via the float$\rightarrow$orderable-uint bit-flip trick [24], [25]; IEEE 754-2019 calls it `totalOrder` [26]; on Tensix the *hardware datapath* imposes it, with no key transform on the critical path.

B. Packer primitives

Table I rows 3–5 carry the packer findings. The exponent histogram costs nothing to enable but samples 1 datum in 8 in a fixed positional pattern — where the Wormhole-era documentation says per-datum [23] — so it can miss every extreme value: exact-count selectors built on it are dead on arrival (§III), and only a sampled threshold *seed* survives. The threshold filter splits the same way: a free compare-and-zero for $T \geq 0$, unusable for $T < 0$, leaving the 2-issue SFPU filter as the ISA floor for signed data.

C. Dst window layout and `SFPSWAP` operand order

Reading a running threshold out of the engine's Dst-resident window (§IV) requires knowing where rank r lives. Full-region Dst dumps against distinct-monotone input, checked word-for-word at $M{=}512/1024/2048$, fix the law (Table I): column-major over $M/16$ physical rows. An a-priori derivation of this layout was wrong; calibration replaced it. A decision tracer that accumulated $-\infty$ on first bring-up fixed the companion fact: `SFPSWAP` with mod1$=$1 routes max to VC — the in-tree comment reads backwards; the vendor functional model agrees [23]. These two facts make §IV's chunk-skip rendezvous implementable.

D. Measured floors and the co-issue law

Four floors bound data-dependent designs (Table I, rows 8–9): counting at 2.0000 cyc/vector exactly; the fp32 unpack-to-dest floor at 3.938 cyc/vector (distinct from the filter bench's 3.855 local floor); a rendezvous at 81/101/98 cyc (`tensix_sync` / semaphore / Dst-sentinel poll), provided the SFPU folds lanes first, since raw-lane reads cost 756–773 cyc. §III prices counting designs with exactly these constants. `SFPLOADMACRO` explains why the count rate is a law: Load$+$Simple$+\{$MAD $\mid$ Store$\}$ maps onto three sub-units that co-issue for free (1.000–1.003 cyc/vector), but reductions need $\geq$2 issues because `SFPIADD` is not macro-schedulable — hence 2 cyc/vector is the counting ceiling. Applied constructively, the same law produced the shipped micro-op wins: merge $1.195/1.221/1.235\times$ and step $1.053/1.074/1.090\times$ at $M{=}512/1024/2048$ (paired MATH_ISOLATE arms, control 2.000 on every run).

VI. EVALUATION

We defend the headline numbers with five exhibits: the 24-cell competition across five measured arms (Table II), the small-k routing before/after (§VI-B), strong scaling against

the cost model (Fig. 3), the chunk-skip A/B (§VI-D), and eight production call-site scenarios (Table III); §VI-F closes by attributing the headline speedup to its parts and pricing the residual gap to the vendor roofline.

A. Harness and Measurement Methodology

All measurements in this paper run on a single Tenstorrent Blackhole p150a (§II-B) under exclusive access. Operator-level timings are device-side kernel durations from the Tracy profiler [27], at operator granularity so host dispatch never contaminates a number. One device process at a time, each sweep cell in its own subprocess: caches, allocator state, and JIT artifacts cannot leak. Kernel-level cycle constants (§V) come from a math-isolated two-point slope over iteration counts, canceling marker overhead; each slope run carries a paired control that must reproduce its known $2.000\times$ ratio or is discarded.

Correctness gates. Every performance cell is gated on bit-exact agreement with a PyTorch golden — never a correlation threshold — so a close-but-wrong kernel cannot post a time. The competition run of Table II recorded 0 wrong results across its 121 gated cells. The surrounding suites hold the same line: a 62-cell numerics contract suite, 154 operator tests, 220/220 routed-path tests, a 36-check adversarial battery, a 2×20-launch hang battery, and 51/51 plus 21/21 bit-exact benchmark cells (counting; RISC scan).

Determinism and provenance. Sweeps are deterministic (`torch.randn` bf16 stimulus, fixed seeds), and every CSV row carries a build fingerprint and source-tree hash — each number attributable to one binary and one tree state.

Trials and spread. Competition cells report medians of 3 trials; chunk-skip cells, 3 trials $\times$ 5 measured iterations with spread below 0.1% everywhere; the single-core baseline sweeps carry standard deviations in-file (e.g., $9{,}492{,}327 \pm 1{,}391$ ns). The anchor cell (k=2048, N=65,536) appears in two independent gated runs — 34.3 µs in the competition sweep, 34.4 µs in the P-sweep re-pin; each is quoted from its own run; the 0.1 µs disagreement bounds run-to-run drift. We state the gap plainly: all results come from one board on one day, and the 121-cell competition run does not commit per-cell standard deviations — a cross-day, higher-trial repeat is scheduled, not done.

Clock caveat. Cycle→microsecond conversions in this paper assume a 1.35 GHz busy clock; the clock under load was not captured (the idle clock measures 800 MHz). Every microsecond figure in this section is a direct Tracy measurement; the caveat applies only where a cycle count is restated in time units, flagged where it occurs.

B. Competition Across Five Measured Arms

Table II sweeps $k \in \{512, 1024, 2048\}$ across $N = 2{,}048$–$262{,}144$ over five measured arms: *pre* — stock `ttnn.topk` before this work; *now* — stock `ttnn.topk` today, including the $\approx 1.17\times$ replay micro-op win that landed in the stock kernels; *routed* — `ttnn.topk` through the new routing (the caller changes nothing); *op* — `topk_large_indices` called directly (columns include the SFPLOADMACRO paths: 3.6% by a disable-flag A/B at the anchor); and *proxy* — the stock bitonic kernels in the operator's row-parallel harness, per-row single-core wall time (it inherits the shared-header micro-op paths: 1.9% by the same A/B). Three headlines. First, the user-facing anchor: at k=2048, N=65,536, the *pre→routed* chain is $631{,}506$ µs → 51.5 µs = $12{,}265\times$ — with no call-site change. Second, the direct-op anchor: the same cell measures **34.3 µs on 32 cores** (8×4 rectangle), an $18{,}401\times$ *pre→op* chain. Third, across all 24 cells the *pre→op* chain spans **632×–51,430×** and *pre→routed* spans 329×–31,905×. The table keeps its own floor honestly: at k=2048, N=2,048 op and proxy tie (15.74 vs. 15.73 µs) — a 2,048-wide row admits no column parallelism.

The routed column rides a TILE-native I/O landing: TILE input by layout dispatch, opt-in TILE output and uint16 indices, deleting the tilize/untilize envelope where k rounds to $\leq$1024 (-67 to -84%); at k=2048 the tilize chain measured cheaper and is kept — per-shape policy by measurement (§VI-F).

The closest internal comparison is a fused attention program embedding a top-k reduction: at its native cell (k=2048, N=65,536) it measures 24.4 µs on 129 cores, $1.4\times$ below the op's 34.3 µs. Both caveats matter: that time *includes* the attention work fused around the selection, and it is a one-call-site fused program, not a callable operator; a per-core breakdown was not committed, so we claim no more than the two end-to-end numbers.

Small-k routing. Against the ≈137 ns/element single-core factory of Section II-C, the routed path wins 40×–423× across the measured before/after pairs ($k \in \{8, 32, 64\}$, W = 5,000–65,536): 695 → 17.2 µs at k=32, W=5,000 (40×); 17.95 ms → 42.4 µs at k=64, W=65,536 (423×). Stock times are 3-trial means (in-file standard deviations ≤ 4.4 µs); routed cells are correctness-gated and bit-stable against the pinned baseline sweep. The same routing removes the $\approx 89\times$ width cliff at the $W < 65{,}535$ bitonic gate (Section II-C).

C. Strong Scaling Versus the Cost Model

Fig. 3 sweeps the slice count P on two cells, 15 measured points total (the factory refuses P=1 — single-core service is the row-parallel path's job). At k=2048, N=65,536 the op falls 183.0 → 34.4 µs from P=2 to P=32; at k=512, N=262,144 it falls 560.2 → 23.3 µs from P=2 to P=104. Both curves are monotone in P (§IV), and the k=512 curve flattens as P approaches the 130-core grid. L1 never bounds P: per-slice fetches are 2–8 KB and the read stream sits 10–100× below op time.

The overlay is §IV's cost model times its two measured merge-unit constants — no further fitting. It tracks the k=2048 curve within 4.3% at every point and predicts the non-monotone bump at P=24: $\lceil 32/24 \rceil = 2$ leaf chunks plus a fifth tree level. On k=512 it lands within 0.3% at the chosen P=104 and overestimates only the deep-serial tail the factory never picks (+33% at P=2, Fig. 3) — the model is accurate where it decides.

TABLE II
COMPETITION ACROSS FIVE MEASURED ARMS, 24 CELLS.

		stock `ttnn.topk` (ms)		routed	op	proxy†	roofline	op gap	pre→routed	pre→op
k	N	pre	now	µs (P)	µs (P)	µs	µs			
512	2,048	4.4	3.7	13.4 (4)	5.9 (4)	10.0	0.32	18.4×	329×	742×
	4,096	9.5	8.0	14.3 (8)	7.1 (8)	18.7	0.46	15.6×	659×	1,327×
	8,192	19.6	16.5	16.5 (16)	8.4 (16)	37.1	0.59	14.2×	1,183×	2,321×
	16,384	39.8	33.6	22.1 (32)	9.7 (32)	71.9	0.73	13.3×	1,799×	4,098×
	32,768	80.2	67.7	19.8 (64)	11.0 (64)	141.4	0.88	12.6×	4,050×	7,284×
	65,536	161.6	137.7	20.9 (64)	13.2 (64)	280.6	1.12	11.7×	7,732×	12,279×
	131,072	323.9	275.8	30.0 (88)	17.4 (88)	566.5	1.56	11.1×	10,806×	18,569×
	262,144	648.4	552.3	45.2 (104)	23.3 (104)	1,008.1	2.39	9.8×	14,334×	27,817×
1024	2,048	7.5	6.3	21.3 (2)	9.4 (2)	11.7	0.45	21.1×	352×	795×
	4,096	17.6	14.9	22.9 (4)	11.4 (4)	21.3	0.64	18.0×	768×	1,543×
	8,192	37.8	31.9	25.6 (8)	13.3 (8)	41.5	0.82	16.2×	1,479×	2,833×
	16,384	78.1	65.9	31.6 (16)	15.2 (16)	79.8	1.01	15.0×	2,473×	5,145×
	32,768	158.8	133.9	30.6 (32)	17.2 (32)	156.5	1.22	14.1×	5,198×	9,234×
	65,536	320.8	273.1	30.2 (64)	19.7 (64)	309.9	1.56	12.6×	10,621×	16,281×
	131,072	644.2	548.2	38.6 (64)	23.5 (64)	616.7	2.17	10.8×	16,705×	27,461×
	262,144	1,291.0	1,098.6	55.5 (88)	32.1 (88)	1,230.2	3.32	9.7×	23,238×	40,199×
2048	2,048	10.0	8.4	28.8 (130)	15.7 (130)	15.7	0.53	29.9×	346×	632×
	4,096	30.1	25.4	32.5 (2)	19.6 (2)	24.6	0.75	26.2×	927×	1,535×
	8,192	70.3	59.4	37.4 (4)	22.9 (4)	47.7	0.97	23.6×	1,878×	3,074×
	16,384	150.8	127.3	45.3 (8)	26.4 (8)	91.8	1.19	22.2×	3,329×	5,705×
	32,768	311.8	263.3	47.6 (16)	30.2 (16)	180.2	1.43	21.1×	6,548×	10,311×
	65,536	631.5	539.5	51.5 (32)	34.3‡ (32)	357.0	1.84	18.7×	12,265×	18,401×
	131,072	1,273.7	1,089.7	61.5 (64)	39.7 (64)	710.5	2.56	15.5×	20,705×	32,092×
	262,144	2,557.6	2,188.1	80.2 (64)	49.7 (64)	1,417.4	3.90	12.8×	31,905×	51,430×

Tracy device kernel durations, medians of 3 trials, every cell bit-exact against a PyTorch golden (0 wrong of 121); arms as defined in §VI-B; the routed column is re-measured on the final tree after the TILE-native landing (same harness and gates, 5-iteration medians, 0 wrong). Roofline: the vendor's published model [28]. †Per-row single-core wall time (arm defined in §VI-B); a proxy, not a shipping configuration. ‡A fused attention+top-k program measures 24.4 µs on 129 cores at this cell (caveats in §VI-B).

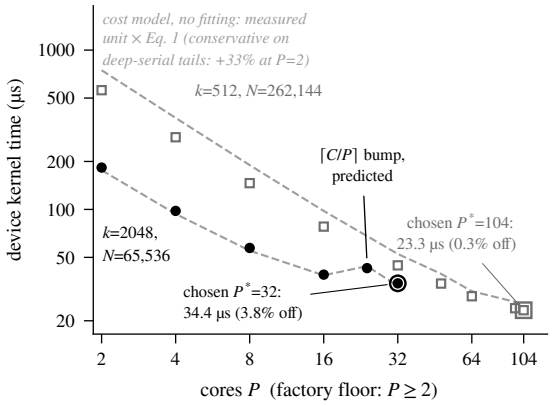

Fig. 3. Measured P-sweep against the unfitted cost model.

D. Chunk-Skip A/B

The row-parallel chunk-skip of §IV was A/B-measured on five cells, three arms each (baseline, ungated, gated). On the shapes the small-k routing ships here, the gated skip wins $\mathbf{279.14} \rightarrow \mathbf{153.19}$ µs (-45.1%, $1.82\times$) at rows=2, k=32, N=65,536, and $\rightarrow$ 177.94 µs (-36.3%) at rows=8 — the rows=8 win is smaller because the op time is the slowest of eight i.i.d. rows. On the three cells where the skip law says no win exists, the gate holds the line: $+0.04\%$ to $+0.51\%$, the largest being gate-off code growth, not the skip test. The column-parallel guard cell re-measures at 13.15 µs with byte-identical kernels.

These measurements close the forecast's loop (§IV-E). The law itself is now confirmed on-device: compile-time-gated skip telemetry — its disabled path proven byte-identical by ELF-disassembly diff — recorded 400 i.i.d. rows at k=32, N=65,536 (128 chunks); per-position skip rates track $e^{-K/(c+1)}$ at all 120 tested positions (the dots of Fig. 2), and rows average 65.11 skips against the law's 66.62 (-2.3%). A gate-constant ablation closes the $K/4$ choice: divisors 2/4/8 move the k=32 cells by at most -0.8% while $/8$ regresses k=512 by $+3.2\%$ — ties where Eq. 2 predicts them — so $K/4$ stands.

E. Production Call-Site Scenarios

Table III replays eight production call-site shapes on the same gated harness — the data contract's three consumer classes (§II-A). The wins concentrate where production is single-core today. The Qwen3 shape that opened the paper improves from its 9,586.9 µs single-core chain to 213.6 µs routed and 193.6 µs direct (44.9–49.5×); split-vocab sampling (deliberately unpadded, N=64,128) behaves the same: 8,924.0 $\rightarrow$ 210.9 µs routed, 188.0 µs direct.

The TP=8 shape already rides the multi-core bitonic: routed measures $1.00\times$ — the honest already-fast row (the direct op still gains $1.4\times$). The sparse-attention indexer rows already call the op by name — today *is* the op — and forcing them through generic routing would cost 22–26%. Their system context is measured too: top-k is $\sim$0.3% of this family's end-to-end prefill — one same-layer collective outweighs the whole cell — so the live lever here is eliding the regather/re-partition inverse pair that brackets the op. The MoE gate rows were shapes the routing predicate declined at the study's

start (the $W \geq 4{,}096$ floor); a measured eligibility carve-out ($k \leq 16$, padded $W \leq 512$, $\leq$32 rows) now captures them automatically: $24.2 \rightarrow 8.18\,\mu s$ and $77.4 \rightarrow 8.34\,\mu s$ ($2.96\times$/$9.29\times$) with zero model-code changes.

One caveat is mandatory: the production sampling call sites pass pre-allocated index tensors, sub-core grid pinning, and `stable=True` — each independently disqualifies routing as-is. The routed column is the *canonical* call form with those arguments dropped — a one-line call-site change whose bit-exactness §VI-F audits, not a free win — and the end-to-end tokens/s effect of that change is unmeasured.

F. Attribution: Where the Speedup Comes From

Assembled, the exhibits answer the question a four-orders-of-magnitude claim owes its reader: what got faster? *Mostly the routing.* The anchor and the production sampling shapes all fail the incumbent's eligibility gates (§II-C) and ran the single-core catch-all while 129 cores idled. Along Table II's own arms, the anchor's $18{,}401\times$ *pre→op* chain factors exactly: $1.17\times$ from micro-op wins inside the stock kernels (§V-D); $1{,}511\times$ available on a *single* core — the proxy arm measures 357.0 µs against stock's 539.5 ms, an algorithm the incumbent shipped but declined to route this shape to; $10.4\times$ from the tree on 32 cores. Escape from the fallback, not a magic kernel, carries the orders of magnitude; the honest apples-to-apples, Table III's TP=8 row where the stock multi-core engine *is* eligible, reads $171.2 \rightarrow 120.8\,\mu s$, $1.4\times$. Core-for-core the honesty holds at small k too: one row-parallel core selects top-32 of 65,536 in $\approx$194 µs wall against the stock factory's $\approx$300 µs per row ($9{,}586.9/32$, Table III) — the $49.5\times$ there is $32\times$ parallelism $\times$ $\approx 1.5\times$ per-core algorithm, a factor that grows to the $1{,}511\times$ above as the factory's cost explodes with k.

That $1.4\times$ — and the parallel term, $10.4\times$ at the anchor, growing to $43\times$ at k=512, N=262,144 — is the algorithmic content, each term separately measured: the log-tree of Algorithm 1 over the serial $P-1$ chain, 1.66–2.01$\times$ at fixed core counts plus monotonicity in P (§IV-A); the cost model with its rectangle embedding, 8–27% (Fig. 3, §IV-B); the law-gated chunk skip, -45.1%/-36.3% on the long streams it opens for and, by Eq. 2, provably ≈ 0 on the 1–5-chunk streams the column split produces — hence the gate (Fig. 2, §IV-D). The parallel term carries its applicability boundary: the tree converts cores into *latency* at ≈ 1.0 core-efficiency — P=2 costs 183.0 µs (§VI-C), 366 core-µs per row vs the shipped kernel's 357.0 on one core ($+2.5\%$) — so the $10.4\times$ serves latency-bound low-row callers, while callers with rows $\geq$ grid are core-optimal on one-core-per-row *except* for the idle remainder wave, which a hybrid landing now harvests: rows past the full row-parallel waves run as concurrent per-row merge trees on the cores the last wave would idle, inside the op (no call-site change). The 160-row indexer of Table III — $712.3\,\mu s \approx 2 \times 357.0$ at the table's pinning — measures 467.0 µs on the final tree ($1.53\times$; the k=512 sibling $559.5 \rightarrow 358.3\,\mu s$, $1.56\times$). Cutting per-core work is the fusion lever below. One term is structural: selection, never sorting — only M-wide candidate windows cross the NoC, $P-1$ in total (Fig. 1, §IV-A). Why these moves and not GPU-style counting is §III's verdict: the machine sells oblivious comparison cheaply and decisions affordably but materialization dearly, so a parallelized comparison network — work skipped, never data compacted — is the architecturally consistent primitive.

What the design does *not* win on is equally measured. The last two data columns of Table II hold the op against the vendor's published roofline, with its k-dependent multipliers (0.612/0.850/1.000) [28]: $9.7\times$–$29.9\times$ across all 24 cells, shrinking monotonically with N at every k. Its contributors are measured elsewhere (§III–§V); whether the roofline is attainable at all for comparator-based kernels — or sits below their critical-path floor — is a derivation we do not print here [MISSING-EVIDENCE: roofline-v2 derivation artifact (G6)]. The measured levers that remain are not smarter compare kernels — §V-D's micro-op audit recovered 5–24% there. One named lever is now banked: the routed$-$op difference was the launch-and-layout envelope — 46% of user-facing time at the anchor — and the TILE-native landing deleted it (§VI-B), cutting the anchor $63.4 \rightarrow 51.5\,\mu s$ (-19%); the 17.2 µs residual is no longer a conversion chain — 9.8 µs is op-internal (the staged TILE-input read) plus the 7.4 µs tilize pair the per-shape policy kept. The lever still open is fusion: the fused attention+top-k program's 24.4 µs (§VI-B, caveats there) is the existence proof that amortizing the launch into the producer buys the next $1.4\times$.

The routing layer, finally, is part of the design space — the arm with the highest measured return per line of code. Table III's model-level sampling wins ($44.9\times$ and $42.3\times$, paid every decoded token) required zero kernel work: an engine-eligibility routing arm plus dropping the three optional call-site arguments of §VI-E; the MoE-gate carve-out repeats the pattern with zero call-site changes ($2.96\times$/$9.29\times$). The drop is audited, not assumed: values bit-exact at the affected shapes, all 27,756 differing index positions across 3,200 audited rows proven bf16 ties, the power-of-two control bit-identical.

VII. Related Work

Deltas by contribution; inline citations appeared in context.

a) Mesh selection theory: Krizanc and Narayanan solved rank selection on abstract meshes three decades ago: optimal $n \times n$-mesh algorithms [11], a deterministic $1.45n$-step bound (sorting needs $2n + o(n)$) [12], and the multi-packet regime our $\lceil C/P \rceil$ leaf term inhabits [21]. The theory assumed unit-cost neighbor communication and returned the k-th element alone: no values-and-indices top-k set, no real NoC costs, no measurement. We therefore claim the first *measured, cost-modeled* full top-k on commercial NoC-mesh silicon. Fagin-style distributed threshold algorithms [29], [30], [31] merge per-node top-k lists — the same tournament at datacenter round-trip costs.

b) GPU selection: The GPU lineage runs from Alabi et al. [2] through WarpSelect [18], Dr. Top-k [3], the SC'23 study [17], and RadiK [4], with database-side [32] and row-wise [33] variants; TPU-KNN [9] and two-stage schemes [10]

TABLE III
PRODUCTION CALL-SITE SCENARIOS WITH BUILT-IN CONTROLS.

call site	rows	N	k	today µs (P)	routed µs (P)	op µs (P)	today→routed	today→op
Qwen3 decode sampling, TP=4 (pad)	32	65,536	32	9,586.9 (1)	213.6 (130)	193.6 (130)	44.9×	49.5×
decode sampling, TP=8 (pad, pow-2)	32	32,768	32	171.2 (65)	171.2 (65)	120.8 (130)	1.00×	1.4×
Llama-3.2 split-vocab sampling	32	64,128	32	8,924.0 (1)	210.9 (130)	188.0 (130)	42.3×	47.5×
DeepSeek-V3.2 sparse-attn indexer	160	65,536	2,048	712.3 (130)	895.3 (130)	712.3 (130)	0.80×	1.00×
DeepSeek-V4 sparse-attn indexer	160	65,536	512	559.5 (130)	682.2 (130)	559.5 (130)	0.82×	1.00×
MiniMax MSA block selection	1	8,192	16	8.4 (16)	114.2 (65)	8.4 (16)	0.07×	1.00×
MoE expert gate, top-4 of 128	32	128	4	8.18 (130)*	8.18 (130)	4.07 (130)	1.00×	2.0×
MoE gate fallback, top-10 of 512	32	512	10	8.34 (130)*	8.34 (130)	4.04 (130)	1.00×	2.1×

All rows Tracy-measured, provenance-stamped, iteration counts recorded. "today" is the engine production selects now: single-core linear (rows 1, 3), 65-core stock bitonic (rows 2, 6), the op itself (rows 4–5, chunk skip live), or the routed gate carve-out (rows 7–8). Op cells include the values output (verified elementwise): $\leq$0.2% over indices-only at sampling shapes, $\approx$6% at the tiny gate shapes. Core counts are launched grids ($\min(\text{rows}, 130)$ active on the row-parallel path). *Routed automatically by the mid-study gate carve-out (k rounds up to the op's 16 floor and slices); previously single-core at 24.2/77.4 µs (§VI-E).

take the approximation escape hatch instead. None of this lineage was ever re-costed on a machine without atomics or scatter — that re-costing is Section III.

c) Running-threshold pruning: WarpSelect discards elements below a per-lane running k-th value before they touch its bitonic queues [18] — element-granular, with no analytic skip law. GVR predicts the threshold from the previous decode step and must verify and refine wrong guesses; it reports 1.88× over TensorRT-LLM's radix select on NVIDIA Blackwell [7]. Chunk-skip (Section IV) skips at chunk granularity, is sound by construction — no verification pass exists to run — and its distribution-free law (Eq. 2) yields the compile-time gate and pre-build forecast. SpAtten's quick-select engine [34] and Focus's streaming concentrator [35] are hardware threshold-engine cousins. The rank statistics are textbook [36]; we find no published chunked corollary, compile-time gate, or soundness proof for skipping inside a bitonic cascade.

d) Accelerator characterization: Published Tensix studies characterize performance — stencils on Grayskull [37], matmul [38]; FFT [39], stencils [40], numerical kernels [41], operator fusion [42], and N-body [43] on Wormhole. An informal 2025 Blackhole microbenchmark report has a dead link and no archive [44]. None covers datapath numerics or selection primitives; Section V owns that delta.

e) The class-wide gap: We find no sorting or selection publication for Cerebras WSE, Groq, SambaNova, Occamy [45], Manticore [46], or Esperanto silicon; Graphcore ships `popops::TopK` with no published algorithm or performance study [47]. This paper prices the ingredients the dataflow-mesh class must pay — but fills the gap for one member.

VIII. METHODOLOGY: AN AGENTIC DESIGN-SPACE CAMPAIGN

An LLM-agent campaign with human review produced these measurements. Prior agentic kernel work optimizes *given* kernels against benchmark suites [48], [49], [50], [51], [52]; this campaign ran a design-space study with measured no-gos on novel silicon; we claim no first-ness, only the record.

The campaign ran as research → implement → validate gates with pre-registered stop rules — a contract gate, a materialization gate (no emit path with headroom ⇒ stop), and an engine shootout; the radix track died by its own rule (Section III-D).

The most transferable instrument was forecast-before-build (Section IV-E): the no-go'd variant was never built (§VI-D).

Agent-generated numbers earn trust only under adversarial audit: one re-verified the working notes' citations (≈55 of 60 exact, none fabricated), re-adjudicated disputed figures, and caught a faster-and-wrong kernel class with schedule-mutation controls — the reward-hacking failure mode [49]. Correctness gates make the audit decidable: every cell fails closed against a bit-exact golden (§VI-A). The campaign also surfaced, and reported upstream, two vendor perf-harness bugs.

The process also failed in ordinary ways: the a-priori Dst-layout derivation was wrong until calibration dumps replaced it (§V-C), and one working-note figure was invented outright until the audit deleted it; neither reached a measured claim.

IX. CONCLUSION

We measured exact top-k across its design space on a Blackhole p150a and closed a four-orders-of-magnitude gap: 631.5 ms → 51.5 µs (12,265×) at the anchor shape, 23.3 µs at the 1M-context shape, 1.82× more from forecast-selected chunk-skipping — attributed honestly in §VI-F: routing supplies the orders of magnitude; the algorithm, 1.4× over the strongest eligible baseline plus its tree, placement, and skip terms. The design-space answer: on a NoC mesh without scatter, the winning economics are parallelized comparison networks (A) plus sound rank-statistic skipping (weak-form (D)) — Floyd–Rivest's lesson [19], not radix's.

The negative result ships with expiry conditions — the counting family (B) reopens if any one holds: (1) a device-side compaction or gather primitive appears, or a composed producer+consumer compressed-stream benchmark proves $\geq$ 2× headroom at k=2048, $N \geq$ 262,144; (2) k outgrows the 2048-element LLK ceiling, ending counting-pass additivity; (3) the consumer becomes a sampler that never materializes the top-k set; (4) N grows until full-data passes (C) dominate. Future work: a Wormhole port, energy, condition (1)'s benchmark, and tokens/s at the relaxed call sites.

REFERENCES

[1] J. Vasiljevic and D. Capalija, "Blackhole & TT-Metalium: The standalone AI computer and its programming model," in *Proc. IEEE Hot Chips 36*, 2024.

[2] T. Alabi, J. D. Blanchard, B. Gordon, and R. Steinbach, "Fast k-selection algorithms for graphics processing units," *ACM Journal of Experimental Algorithmics*, vol. 17, 2012.

[3] A. Gaihre *et al.*, "Dr. top-k: Delegate-centric top-k on GPUs," in *Proc. SC*, 2021, arXiv:2109.08219.

[4] "RadiK: Scalable and optimized GPU-parallel radix top-k selection," in *Proc. ACM ICS*, 2024, arXiv:2501.14336. UNVERIFIED: verify authors.

[5] A. Fan, M. Lewis, and Y. Dauphin, "Hierarchical neural story generation," in *ACL*, 2018, introduces top-k sampling for neural text generation. arXiv:1805.04833.

[6] DeepSeek-AI, "Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention," DeepSeek-AI, Tech. Rep., 2025, sparse-attention indexer selecting top-2048 keys per layer. [verify exact title/identifier before camera-ready].

[7] "Guess-verify-refine: Data-aware top-k for sparse-attention decoding on Blackwell via temporal correlation," *arXiv preprint*, 2026, arXiv:2604.22312. UNVERIFIED: verify authors.

[8] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *ICLR*, 2017, noisy top-k expert gating. arXiv:1701.06538.

[9] F. Chern, B. Hechtman, A. Davis, R. Guo, D. Majnemer, and S. Kumar, "TPU-KNN: K nearest neighbor search at peak FLOP/s," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, uNVERIFIED: verify author list.

[10] "A faster generalized two-stage approximate top-k," *arXiv preprint*, 2025, arXiv:2506.04165; OpenReview izqZ1Crpjz. UNVERIFIED: verify authors.

[11] D. Krizanc and L. Narayanan, "Optimal algorithms for selection on a mesh-connected processor array," in *Proc. IEEE Symposium on Parallel and Distributed Processing (SPDP)*, 1992.

[12] "Fast deterministic selection on mesh-connected processor arrays," *Algorithmica*, 1.45n steps on an $n \times n$ mesh vs $2n + o(n)$ sort-based; earlier version LNCS, doi 10.1007/3-540-54967-6_79. UNVERIFIED: verify authors/year.

[13] Intel Corporation, "x86-simd-sort: SIMD-accelerated sorting and partial sorting," https://github.com/intel/x86-simd-sort, 2023, uNVERIFIED: verify release/year before camera-ready.

[14] M. Blacher, J. Giesen, P. Sanders, and J. Wassenberg, "Vectorized and performance-portable Quicksort," *Software: Practice and Experience*, 2023, arXiv:2205.05982. UNVERIFIED: verify authors/venue/volume before camera-ready.

[15] Tenstorrent, "TT-NN operator reference: `ttnn.topk`," TT-NN documentation, 2026, accessed 2026-08-16.

[16] ——, "`topk_local_sort` bitonic top-k kernel discussion," tt-metal issue tracker, #33492, engineering artifact; documents the bitonic local-sort top-k path. [verify year/permalink before camera-ready].

[17] "Parallel top-k algorithms on GPU: A comprehensive study and new methods," in *Proc. SC*, 2023, uNVERIFIED: verify authors before camera-ready.

[18] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, 2019, warpSelect. arXiv:1702.08734.

[19] R. W. Floyd and R. L. Rivest, "Expected time bounds for selection," *Communications of the ACM*, vol. 18, no. 3, pp. 165–172, 1975.

[20] N. Samardzic, W. Qiao, V. Aggarwal, M.-C. F. Chang, and J. Cong, "Bonsai: High-performance adaptive merge tree sorting," in *Proc. ACM/IEEE ISCA*, 2020.

[21] "Multi-packet selection on mesh-connected processor arrays," in *Proc. International Parallel Processing Symposium (IPPS)*, 1992, uNVERIFIED: verify authors.

[22] Z. Jia, B. Tillman, M. Maggioni, and D. P. Scarpazza, "Dissecting the Graphcore IPU architecture via microbenchmarking," Citadel, Tech. Rep., 2019, arXiv:1912.03413.

[23] Tenstorrent, "Tensix ISA documentation (Blackhole and Wormhole)," Vendor ISA documentation, 2025, `Dst`, `SFPGT`, `SFPSWAP`, `SFPLOADMACRO`, and packer exponent-histogram sections; the Wormhole packer-histogram page documents per-datum counting, contradicted on Blackhole by the measurements of §5 (1-in-8 positional sampling).

[24] M. Herf, "Radix tricks," http://stereopsis.com/radix.html, 2001.

[25] D. Merrill and A. Grimshaw, "High performance and scalable radix sorting," *Parallel Processing Letters*, vol. 21, no. 2, 2011.

[26] IEEE, "IEEE standard for floating-point arithmetic (IEEE 754-2019)," IEEE Std 754-2019, 2019, totalOrder predicate, §5.10.

[27] B. Taudul, "Tracy: A real-time, nanosecond-resolution frame profiler," https://github.com/wolfpld/tracy, 2024, device-side timeline capture as integrated in the vendor's Metalium runtime.

[28] Tenstorrent, "LLM performance model: operator roofline tables," Tenstorrent `llm_perf` performance-model repository, 2026, roofline tables for Blackhole with top-k multipliers 0.612/0.850/1.000 for k=512/1,024/2,048. UNVERIFIED: pin exact revision before camera-ready.

[29] R. Fagin, A. Lotem, and M. Naor, "Optimal aggregation algorithms for middleware," *Journal of Computer and System Sciences*, vol. 66, no. 4, 2003.

[30] P. Cao and Z. Wang, "Efficient top-k query calculation in distributed networks," in *Proc. ACM PODC*, 2004.

[31] S. Michel, P. Triantafillou, and G. Weikum, "KLEE: A framework for distributed top-k query algorithms," in *Proc. VLDB*, 2005. [Online]. Available: https://www.vldb.org/archives/website/2005/program/paper/thu/p637-michel.pdf

[32] A. Shanbhag, H. Pirk, and S. Madden, "Efficient top-k query processing on massively parallel hardware," in *Proc. ACM SIGMOD*, 2018.

[33] "RTop-K: Ultra-fast row-wise top-k selection for neural network acceleration on GPUs," *arXiv preprint*, 2024, arXiv:2409.00822. UNVERIFIED: verify authors/venue.

[34] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient sparse attention architecture with cascade token and head pruning," in *Proc. IEEE HPCA*, 2021, arXiv:2012.09852.

[35] "Focus: A streaming concentration architecture for efficient vision-language models," *arXiv preprint*, 2025, arXiv:2512.14661. UNVERIFIED: verify authors; read full text (closest architecture-paper analog to a chunked running-threshold engine).

[36] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.

[37] N. Brown and R. Barton, "Accelerating stencils on the Tenstorrent Grayskull RISC-V accelerator," in *Proc. SC Workshops*, 2024, arXiv:2409.18835.

[38] "Assessing Tenstorrent's RISC-V MatMul acceleration capabilities," *arXiv preprint*, 2025, arXiv:2505.06085. UNVERIFIED: verify authors.

[39] N. Brown, J. Davies, and F. Le Clair, "Exploring fast Fourier transforms on the Tenstorrent Wormhole," in *Proc. ISC High Performance Workshops*, 2025, arXiv:2506.15437. UNVERIFIED: verify given names.

[40] "Stencil computations on Tenstorrent Wormhole," *arXiv preprint*, 2026, arXiv:2605.07599. UNVERIFIED: verify authors.

[41] "Numerical kernels on a spatial accelerator: A study of Tenstorrent Wormhole," *arXiv preprint*, 2026, arXiv:2603.23343. UNVERIFIED: verify authors.

[42] "Operator fusion for LLM inference on the Tensix architecture," *arXiv preprint*, 2026, arXiv:2606.09879. UNVERIFIED: verify authors.

[43] Amati *et al.*, "Accelerating gravitational n-body simulations using the RISC-V-based Tenstorrent Wormhole," in *Proc. SC Workshops*, 2025, arXiv:2605.02744 / 2509.19294. UNVERIFIED: verify full author list.

[44] "Dissecting the Tenstorrent Blackhole architecture via microbenchmarking," Informal technical report, Tech. Rep., 2025, uNVERIFIED: formerly at asplos.dev (404 as of 2026-08-16); locate archived copy before submission.

[45] "Occamy: A 432-core dual-chiplet dual-HBM2E 768-DP-GFLOP/s RISC-V system for 8-to-64-bit dense and sparse computing in 12nm FinFET," *arXiv preprint*, 2025, arXiv:2501.07330. UNVERIFIED: verify authors.

[46] F. Zaruba, F. Schuiki, and L. Benini, "Manticore: A 4096-core RISC-V chiplet architecture for ultraefficient floating-point computing," *IEEE Micro*, vol. 41, no. 2, 2021.

[47] Graphcore, "TopK — Poplar and PopLibs API reference (popops), v3.1.0," https://docs.graphcore.ai/projects/poplar-api/en/3.1.0/poplibs/popops/TopK.html, 2023.

[48] A. Ouyang *et al.*, "KernelBench: Can LLMs write efficient GPU kernels?" *arXiv preprint*, 2025, arXiv:2502.10517. UNVERIFIED: verify author list.

[49] Sakana AI, "The AI CUDA engineer," Sakana AI, Tech. Rep., 2025, includes the reward-hacking cautionary tale motivating verification discipline.

[50] "AccelOpt: A self-improving LLM agentic system for AI accelerator kernel optimization," *arXiv preprint*, 2025, arXiv:2511.15915. UNVERIFIED: verify authors.
[51] "KForge: Program synthesis for diverse AI hardware accelerators," *arXiv preprint*, 2025, arXiv:2511.13274; see also arXiv:2606.02963. UNVERIFIED: verify authors; check target list for Tenstorrent.
[52] "PEAK: A performance engineering AI-assistant for GPU kernels powered by natural language transformations," *arXiv preprint*, 2025, arXiv:2512.19018. UNVERIFIED: verify authors.